Introduction to Skills

This article provides an overview of developing skills for Misty. It describes how Misty works with your code, and it explores some of the differences between developing for Misty and developing for other platforms. It also provides guidelines for sharing skills and collaborating with other developers in the Misty Community.

What is a Skill?

At a high level, there are two ways to code your Misty II:

- You can build a **skill** using Misty's JavaScript or .NET SDK. A skill runs directly on the robot, alongside Misty's own software.
- You can build a robot **application** using Misty's REST API. The distinction between an application and a skill is that the code for an application runs on an external device (like your desktop browser or on a Raspberry Pi) and not onboard the robot.

While you can build many different types of skills and robot applications, most involve some combination of the following:

1. gathering some of Misty's sensory data
2. processing that data using Misty's API, or sending it off to a cloud service
3. having Misty do something in response

For example, you might code Misty to do the following:

- roam an office to greet people she recognizes
- respond with sound and movement when you touch her on the head or chin
- patrol an area to capture photos, videos, and audio recordings while reporting changes in the environment

The first step in coding Misty is deciding what you want the robot to do. Then, you can choose to whether to use Misty's JavaScript SDK, her .NET SDK (Beta), or her REST API. When you're ready, you can share your code with other developers in the Misty Community.

SDKs and APIs

Misty's JavaScript, .NET, and REST APIs include many of the same commands, but they differ in architecture and implementation. The following sections provide a high-level overview of Misty's SDKs and APIs.

JavaScript SDK

When you build a skill with Misty's JavaScript SDK, your code runs locally on your robot. Each JavaScript skill runs from a single JavaScript code file. You upload this code file to Misty along with a .json meta file that defines some of the initial settings and parameters that Misty will use when running the skill. If the skill uses image or audio assets that don't yet exist on the robot, you can upload these assets at the same time you upload your skill code. Learn more about this process in the File Structure & Code Architecture section.

> **Note:** If you're used to writing JavaScript for the web, you may be familiar with manipulating the DOM in your code. With Misty, there is no DOM for your code to manipulate. Skills you write with Misty's JavaScript SDK cannot use the various objects and data types present in the DOM.

The syntax for using a command in Misty's JavaScript API is:

```
misty.<COMMAND>(parameters);
```

Js

So, for example, to change the color of Misty's logo LED, you would call the `misty.ChangeLED()` command like this:

```
misty.ChangeLED(255, 0, 0);
```

Js

In addition to methods for using Misty's robot capabilities, the on-robot JavaScript API includes methods for:

- sending debug messages
- saving data to Misty
- handling data from sensor and skill events
- controlling the flow of your code's execution
- sending external requests to third party web APIs
- starting and stopping the execution of other JavaScript code files on Misty

Follow these links to learn more about coding with Misty's JavaScript SDK:

- Architecture
- Tutorials
- Reference Documentation

Code you write with Misty's JavaScript SDK executes locally on the robot. This means you can expect Misty to respond to commands immediately after they execute in your code. This differs from using Misty's REST API, where you can expect some latency between the execution of a command and Misty's response.

> **Tip:** Writing skills that can run entirely on Misty offers portability and allows you to easily package and redistribute skills for reuse. Being able to encapsulate and reuse skills is the key to having a robot that does many tasks.

Consider using Misty's JavaScript SDK when:

- you want your skill to be portable, so that Misty can run it without depending on or requiring an external device
- you want to be able to package and share your functionality with other developers or Misty owners
- you need Misty to respond to commands immediately after they execute in your code
- your skill does not require input from another interface (like text from a computer)

.NET SDK

Misty II's .NET SDK provides tools for building, debugging, and deploying skills with Microsoft Visual Studio and the .NET framework. Skills you build with the .NET SDK assemble into background applications that run alongside Misty's software on Windows IoT Core. With Misty's .NET SDK, you can:

- Write skills in C#
- Leverage features of Visual Studio and the .NET framework such as autocomplete, IntelliSense, and live remote debugging
- Use external libraries in your skill code
- Test code before deployment by simulating input from sensors and other robot data

You build a .NET skill for Misty by wrapping your C# skill code in a background application and deploying the application to Misty's Windows 10 IoT Core device. Your skill code implements the interface and methods for issuing commands and getting data from the robot. You can read more about the architecture of a .NET skill in the .NET Skill Architecture documentation.

Currently, Misty's .NET SDK includes the following libraries and templates:

- Misty Skill Extension (Beta) - A Visual Studio extension that contains project templates for C# skills. When you create a new project with this template, Visual Studio automatically installs the Misty Robotics SDK libraries for use in your skill.
- MistyRobotics.SDK (Beta) - Library used to build .NET skills for Misty
- MistyRobotics.Common (Beta) - Library with types and data objects commonly used across Misty's .NET skill projects
- MistyRobotics.Tools (Beta) - Library with optional tools for programming Misty

> **Note:** Comprehensive reference documentation for the public members of Misty's .NET SDK libraries is still under development. For now, we recommend using IntelliSense to browse the contents of the SDK libraries and access summaries of library classes and their members.

In addition to methods for using Misty's robot capabilities, Misty's .NET SDK libraries provide methods for:

- sending debug messages
- saving data to Misty
- handling data from sensor and skill events
- controlling the flow of your code's execution
- starting and stopping the execution of other skills on Misty
- simulating event data and command responses

Follow these links to learn more about coding with Misty's .NET SDK:

- Overview
- Getting Started
- Architecture

Like JavaScript skills, .NET skills run locally on Misty II. Consider using the .NET SDK when:

- you want to leverage the features of Visual Studio and the .NET framework
- you want to use external libraries for advanced local processing of input from Misty's sensors and other event data
- you want your skill to be portable, so that Misty can run it without depending on or requiring an external device
- you want to be able to package and share your functionality with other developers or Misty owners
- you need Misty to respond to commands immediately after they execute in your code
- your skill does not require input from another interface (like text from a computer)

REST API

With Misty's REST API, your code runs on an external device and makes HTTP requests to Misty over your local Wi-Fi connection. You send requests to Misty's REST API the same way you interact with other RESTful web APIs. The code you write to send these requests can execute from a web page, a microcontroller (like an Arduino), or any other device that can send web requests.

Writing a robot application with Misty's REST API typically involves two things:

- getting data from Misty's sensors via WebSocket connections
- sending requests to Misty's REST API endpoints

The structure and syntax for sending requests to Misty's REST API endpoints depends on the programming language and libraries you use to write your skill. For an example of sending requests with JavaScript from your web browser, see Sending Commands to Misty.

Code you write for Misty's REST API runs on a device outside Misty to send requests over your local Wi-Fi network. This can introduce some latency between the moment your skill sends a request and Misty's execution of the associated command.

It also means you can integrate the capabilities of the device from which the code executes into your skill. For example, if you use Misty's REST API to write skill for Misty to learn someone's face, you can design a web interface with a form where this person can type their name and send it to Misty.

Consider using Misty's REST API when:

- you want to write a skill for Misty using a language other than JavaScript or C#
- your skill depends on the integration of a graphic interface, like a web page
- your skill uses the capabilities of an external device that cannot communicate with Misty directly through her built-in UART serial port
- you want to use JavaScript features that are not supported by Misty's JavaScript engine
- your skill uses third-party services not yet supported by Misty's JavaScript SDK

Follow these links to learn more about Misty's REST API:

- Architecture
- Tutorials
- Reference Documentation

Community Resources

You can find helper libraries, code samples, wrappers, and other useful tools for skill development in the Misty Robotics Community on GitHub.

For examples of working skills that use a variety of Misty's capabilities, see Misty Skill Samples on GitHub.

For a collection of skills that other Misty II developers have shared, see the Misty Skills section of the community forums.

Sharing Skills & Applications

When you are ready to share a skill or application with other developers, you can post your code in the Misty Community Forums. Our recommendation for sharing skills and applications is as follows:

- Remove any passwords, credentials, API keys, or tokens your skill uses from your code.
  1. If your code uses 3rd party services, we recommend you create a `credentials.json` file where you can store the relevant details.
  2. To help your fellow developers, annotate your code in the places where the skill or application uses these accounts and credentials. You can list the details and instructions about how to acquire and use credentials in the skill in a `README` file in the same repository as the skill.
- Create a repository that you maintain on GitHub or somewhere else where other developers can download your code.

- License your repository. You can use whichever license you'd like. For more information about choosing a license, see GitHub's article on Licensing a Repository.
- Create a new topic in the Misty Skills category of the Misty Community forums.
- Set the first post in your topic to be a wiki. This will allow you and other contributors to suggest changes to the original post as development on your skill continues.
  1. To set your post to be a wiki, select the three-dots icon from the menu below the post.

     
     Discourse Wik Step 1

  2. Select the wrench icon.

     
     Discourse Wik Step 2

  3. Select the **Make Wiki** option from the menu.

     
     Discourse Wik Step 3

- Update the original post when you make changes to your code.